

LeetCode 875: Koko Eating Bananas

BINARY SEARCH

DE SHAW

DEUTSCHE

ZEPTO

OYO

DELOITTE

Problem Overview

Given n piles of bananas, guards have left for h hours. We need to return k , such that Koko eats k bananas from a pile per hour in order to finish all bananas before the guard arrives.

Example: Piles = [3, 6, 7, 11], $h = 8$

If $k = 4$, then hours taken = $1 + 2 + 2 + 3 = 8 \leq 8$ (which is the minimum valid speed).

Key Intuition: Minimum + Monotonic Function → Binary Search

If a speed k works (meaning she finishes in time), then speeds $k+1, k+2, \dots$ will also definitely work. Since we need to find the smallest valid k , a binary search approach is optimal.

1. Brute Force Approach

Try every possible speed starting from **1** to **mx**, where **mx** is the maximum number of bananas in a single pile.

Time Complexity: $O(mx \times n)$

```
class Solution {
public:
    int minEatingSpeed(vector<int>& piles, int h) {

        int mx = *max_element(piles.begin(), piles.end());

        for(int speed = 1; speed <= mx; speed++){

            long long hours = 0;

            for(int bananas : piles){
                hours += (bananas + speed - 1) / speed;
            }

            if(hours <= h){
                return speed;
            }
        }

        return mx;
    }
};
```

2. Optimal Approach (Binary Search)

Instead of linearly checking speeds, we apply binary search in the search space $[1, mx]$.

```
class Solution {
public:
    bool check(vector<int>& piles, int h, int ans){
        long long hours=0;
        for(int banana : piles){
            hours += ceil(1.0 * banana / ans);
        }

        if(hours <= h){
            return true;
        }
        return false;
    }

    int minEatingSpeed(vector<int>& piles, int h) {
        int l = 1;
        int hi = *max_element(piles.begin(), piles.end());
        int ans = hi;

        while(l <= hi){
            int mid = l + (hi - l) / 2;

            if(check(piles, h, mid)){
                ans = mid;
                hi = mid - 1; // aur chhota answer khojo
            }
            else{
                l = mid + 1;
            }
        }

        return ans;
    }
};
```